

University of ESwatini Online Voting System Project Plan

1. Introduction

1.1 Background and History of the Project

The Online Voting System project was started to address the problems of traditional voting methods within the institution of higher learning. Manual voting system is mostly relying on paper ballots and physical counting of ballots, are time-consuming, exposed to human error, and vulnerable to manipulation and fraud. They also require significant logistic arrangements, which can discourage participation of voters and increase costs. Since advancement of information technology, there is an increase in demand for digital solutions that can simplify the process voting while maintaining integrity and transparency. This project is developed to meet those needs, aiming to provide a secure, efficient, and user-friendly platform that enhances participation and trust in elections.

1.2 Aims and Objectives

The fundamental purpose of the project is to design and implement a secure online voting system that ensures fairness, transparency, and accessibility. The system is intended to serve as a model for university elections. It's objectives are the following:

- To make sure that one person is assigned one vote through robust authentication mechanisms.
- To provide real time summing up of votes and generate accurate results dashboards.
- To reduce administrative costs by automating candidate management and result reporting.
- To enhance voter engagement by allowing participation from any location, thereby removing geographical and time constraints.
- To incorporate strong security measures, including encryption, password hashing, and audit trails, to safeguard against fraud and data breaches.

1.3 Responsible Persons in the Development of the System

The project is organized around the collaboration of different role players making up the team structure, with each member assigned specific responsibilities to make sure of smooth execution:

- *Project Manager*: Oversees the timeline, milestones, and overall coordination of the project.
- *System Analyst*: Gathers requirements from stakeholders and translates them into technical specifications.
- *Developers*: Build both front end and back end components, including authentication, voting, results, and administrative modules.
- *Database Administrator*: Designs and manages the database schema, ensuring data integrity and performance.
- *Testers*: Validate the system through unit, integration, security, and usability testing.
- *Deployment Administrator*: Manages hosting, configuration, and roll out of the system.

1.4 Summary of the Project

The Online Voting System project is composed of six phases: planning and analysis, system design, implementation, testing, deployment, and documentation with presentation. The project spans twelve weeks, with each phase having the previous one as a predecessor. Planning and analysis establish the foundation, while system design translates requirements into technical blueprints. Implementation develops the core modules, followed by intense testing to ensure reliability and security. Deployment prepares the system for real-world use, and documentation ensures that both technical and user perspectives are captured. The project uses Gantt charts to visualize scheduling and PERT charts to show dependencies and the critical path. Slack time is incorporated into tasks such as UI design and documentation, allowing flexibility without delaying completion.

2. User/Client Involvement

The University of Eswatini, as the client, plays a central role in the successful development and deployment of the Online Voting System. Their involvement ensures that the system is aligned with institutional policies, electoral regulations, and the practical needs of students and administrators. The client's contributions are categorized into information, services, resources, and facilities, each provided at specific stages of the project.

2.1 Information Provided by the Client

At the beginning of the project, which is the Planning and Analysis phase, the university provides crucial information including the final voter list, candidate details, and election rules that are found in the Student Union Constitution. This information is required for the requirements gathering and system design. Additionally, the client provides authentication protocols, such as student ID formats and registration procedures, which guide the development of secure login and verification.

2.2 Services Provided by the Client

Throughout the project, the university offers administrative services to support development and testing. During the Implementation phase, administrators assist in validating modules such as candidate management and voting enforcement. In the Testing phase, the client is responsible for supervising simulation of a real election environment, assuring that the system complies with university standards. These services ensure that the system reflects real election scenarios and meets expectations of the user.

2.3 Resources Provided by the Client

The university provides important resources during the Design and Implementation phases. These include access to university databases for integration, sample datasets for testing, and policy documents governing elections. The client also provides resources such as server access credentials and guidelines for data storage, ensuring that the system fits with existing structures. These resources are critical for building a system that works well.

2.4 Facilities Provided by the Client

During the Deployment phase, the university makes available physical and digital infrastructure needed for hosting and demonstration. This includes server space within the university's IT infrastructure, secure network access, and testing laboratories where students and administrators can interact with the system. In the Documentation and Presentation phase, the university provides venues for final demonstrations, such as lecture halls or computer labs, ensuring that stakeholders can evaluate the system in a controlled environment.

3. Risk Identification and Management

Risk management is an important part of the Online Voting System project, making sure that risks that may arise are identified as early as possible and addressed accordingly with appropriate strategies. Given the sensitivity of electoral processes, risks related to security, reliability, usability, and compliance must be anticipated and mitigated to safeguard the integrity of the system.

3.1 Unauthorized Access and Multiple Voting Attempts

One of the most critical risks is the possibility of unauthorized users gaining access to the system or legitimate users attempting to vote more than once. Such breaches could compromise the fairness of the election.

Control Strategy: Strong authentication methods will be implemented, including secure login credentials, password hashing, and optional one-time password (OTP) verification. The system will enforce one person one vote by locking accounts immediately after a vote is cast. Audit logs will track all voting activities to detect anomalies.

3.2 Server Downtime During Peak Voting Periods

High traffic during election hours may overload servers, leading to downtime and preventing voters from accessing the system.

Control Strategy: The system will be hosted on scalable infrastructure, either through cloud platforms or optimized local servers. Load balancing and redundancy will be employed to distribute traffic evenly. Backup servers will be configured to ensure continuity in case of primary server failure.

3.3 Data Privacy Breaches

Sensitive voter information, including student IDs and login credentials, could be exposed if the system is not properly protected.

Control Strategy: All data transmissions will be encrypted using HTTPS protocols. Passwords will be stored using hashing algorithms, and access to the database will be restricted to authorized personnel only. Regular security audits will be conducted to identify vulnerabilities.

3.4 SQL Injection and Other Cyber Attacks

Hackers may attempt to exploit vulnerabilities of the system by inserting malicious SQL code, brute force login attempts, or denial of service attacks.

Control Strategy: Input validation and parameterized queries will be enforced to prevent SQL injection. Intrusion detection systems will monitor suspicious activities, while rate-limiting will reduce the risk of brute force attacks. Firewalls and antivirus software will provide additional layers of protection.

3.5 Usability Issues for Voters and Administrators

If the system is not intuitive, voters may struggle to cast their votes, and administrators may find it difficult to manage candidates and results.

Control Strategy: The UI/UX design will prioritize simplicity and clarity, with wire frames tested during the design phase. Usability testing will be conducted with sample users to ensure that both voters and administrators can interact with the system effectively.

3.6 Incomplete or Inaccurate Requirements Gathering

Failure to capture all functional and non-functional requirements may result in a system that does not meet the university's needs.

Control Strategy: Requirements will be gathered through interviews, surveys, and consultations with stakeholders. Use case diagrams and requirement documents will be validated by the client before design begins. Any changes will be managed through a formal configuration management process.

3.7 Project Timeline Delays

Delays in one phase may interrupt the next phases, jeopardizing the twelve-week schedule.

Control Strategy: The project will be managed using Gantt and PERT charts to monitor progress and dependencies. Slack time will be built into non-critical tasks such as documentation, allowing flexibility without affecting the critical path. Regular progress reviews will ensure adherence to the timeline.

4. Standards, Guidelines, and Procedures

For the successful development of the University of Eswatini Online Voting System, all individuals involved in the project are required to adhere to established standards, guidelines, and procedures. These practices ensure consistency, maintain quality, and safeguard the integrity of the system. By following agreed-upon rules, the project team minimizes risks, improves collaboration, and delivers a system that meets both functional and non-functional requirements.

4.1 Coding Standards

Developers must follow strict coding standards to ensure readability, maintainability, and scalability of the system. Naming conventions for variables, functions, and classes must be consistent across modules. Inline documentation and comments are mandatory to explain logic and facilitate future maintenance. Code must be version-controlled using GitHub or Git Lab, with commits properly labeled to track changes. These standards guarantee that the system remains transparent and manageable throughout its life cycle.

4.2 Security Guidelines

Given the sensitivity of electoral data, security guidelines are important. Passwords must be hashed using secure algorithms, and all communication between client and server must be encrypted via HTTPS. Authentication procedures must enforce one person one vote, with accounts locked after voting. Access to the database is restricted to authorized personnel, and audit logs must be maintained to track all activities. Developers and administrators are required to conduct regular security reviews to identify vulnerabilities and apply patches promptly.

4.3 Testing Procedures

Testers must follow standardized procedures to validate system functionality. Unit testing is conducted on individual modules such as authentication and vote casting, while integration testing ensures seamless interaction between components. Security testing simulates attacks such as SQL injection and brute force attempts, while usability testing evaluates the system's accessibility for both voters and administrators. Test cases must be documented, and results recorded to provide evidence of compliance with quality assurance standards.

4.4 Documentation Standards

Analysts and developers are required to produce clear and comprehensive documentation at each stage of the project. Requirements documents must capture both functional and non-functional needs, while design documents must include ERDs, UML diagrams, and architectural decisions. Implementation documentation should describe module functionality, while testing reports must summarize procedures and outcomes. A user manual must be prepared to guide administrators and voters in operating the system. Documentation ensures transparency and supports future system upgrades.

4.5 Procedural Guidelines for Collaboration

All team members must adhere to agreed collaboration procedures. Weekly progress meetings are scheduled to review milestones and address challenges. Change requests must be logged and approved by the project manager before implementation. Peer reviews are mandatory for code submissions, ensuring that errors are identified early. Communication between team members must be professional, with updates shared through designated project management tools.

5. Organisation of the Project

5.1 Relation to Other Projects

The Online Voting System project is related to other digital initiatives taken by the University of Eswatini. It needs existing systems such as student information management platforms and online portals by integrating authentication protocols and data management practices. The project also aligns with broader institutional goals of digitization, efficiency, and transparency, serving as a model for future projects in areas such as online examinations, e-learning, and digital record-keeping.

5.2 Organisation of the Project Itself

The project is structured into six phases: planning and analysis, system design, implementation, testing, deployment, and documentation with presentation. Each phase builds upon the previous one, ensuring a logical progression from requirements gathering to final delivery. The project is managed using Gantt and PERT charts, which provide clear timelines, dependencies, and critical paths. Slack time is incorporated into non-critical tasks such as documentation and UI design, allowing flexibility without delaying completion. Weekly progress reviews are scheduled to monitor milestones and address challenges promptly.

5.3 Roles of Personnel

The project team is composed of individuals with clearly defined roles to ensure accountability and efficiency:

- a. *Project Manager*: Oversees the timeline, coordinates tasks, manages risks, and ensures milestones are achieved.
- b. *System Analyst*: Gathers requirements from stakeholders, prepares use case diagrams, and translates needs into technical specifications.
- c. *Programmer/Developer*: Implements front end and back end modules, including authentication, voting, results, and administration.
- d. *Designer*: Creates UI/UX wire frames and ensures the system is user-friendly and accessible.
- e. *Database Administrator*: Designs and manages the database schema, ensuring data integrity, security, and performance.
- f. *Tester*: Conducts unit, integration, security, and usability testing to validate system functionality.
- g. *Deployment Administrator*: Manages hosting, server configuration, and roll out of the system.

5.4 Training Requirements

To ensure competence and readiness, personnel involved in the project must undergo targeted training sessions. Training requirements are as follows:

- a. *Project Manager*: Training in project management tools (PERT, Gantt charts, version control systems) and risk management strategies.

- b. *System Analyst*: Training in requirements elicitation techniques, UML modeling, and documentation standards.
- c. *Programmers/Developers*: Training in secure coding practices, back end frameworks (PHP, Node.js, Python), and front end technologies (HTML, CSS, JavaScript).
- d. *Designer*: Training in UI/UX principles, wire framing tools, and accessibility standards.
- e. *Database Administrator*: Training in database design, SQL optimization, and security protocols.
- f. *Tester*: Training in automated testing tools, penetration testing, and usability evaluation.
- g. *Deployment Administrator*: Training in server configuration, cloud hosting platforms, and encryption protocols.

5.5 Training Schedule

Training is scheduled to align with project phases, ensuring personnel are prepared before their responsibilities commence:

- Planning and Analysis: Training for Project Manager and System Analyst on project management and requirements gathering.
- System Design: Training for Designers and Database Administrators on wireframing, ERD creation, and security standards.
- Implementation: Training for Developers on secure coding practices and backend/frontend frameworks.
- Testing: Training for Testers on automated testing tools and penetration testing.
- Deployment: Training for Deployment Administrator on server hosting, cloud platforms, and HTTPS configuration.
- Documentation & Presentation: Training for all personnel on documentation standards and presentation skills.

6. Project Phases

6.1 Choice of Life Cycle Model

The most suitable life cycle model for the University of Eswatini Online Voting System is the Waterfall Model with minor iterative overlaps. This model is appropriate because the project has well-defined requirements, a fixed twelve-week timeline, and clear deliverables. Each phase builds upon the previous one, ensuring structured progression. However, limited overlaps (such as documentation beginning during testing) provide flexibility and reduce risks of delay.

6.2 Phase 1: Requirements Analysis

Tasks: Define project scope, identify stakeholders, gather functional and non-functional requirements, create use case diagrams, and assess security risks.

Milestones: Completion of the requirements specification document and approval by the client.

Verification: Milestone is reached when the University of Eswatini formally validates the requirements document.

Effort Estimate: 2 weeks, approximately 80 staff-hours (analyst and project manager).

6.3 Phase 2: System Design

Tasks: Database design (ERD), system architecture (three-tier model), UI/UX wireframes, technology stack selection, and documentation of design decisions.

Milestones: Finalized ERD, approved architecture diagrams, and design documentation.

Verification: Milestone is reached when design documents are reviewed and signed off by the project manager and client.

Effort Estimate: 2 weeks, approximately 120 staff-hours (analyst, designer, database administrator).

6.4 Phase 3: Implementation

Tasks: Development of authentication, candidate management, voting module, results dashboard, and admin panel.

Milestones: Completion of each module and successful integration into the system.

Verification: Milestone is reached when modules pass initial unit testing and are integrated without errors.

Effort Estimate: 4 weeks, approximately 240 staff-hours (developers and database administrator).

6.5 Phase 4: Testing

Tasks: Unit testing, integration testing, security testing (SQL injection, brute force), and usability testing.

Milestones: Test reports documenting successful validation of all modules.

Verification: Milestone is reached when all test cases pass and client representatives confirm usability.

Effort Estimate: 2 weeks, approximately 100 staff-hours (testers and developers).

6.6 Phase 5: Deployment

Tasks: Hosting on server/cloud, database configuration, HTTPS setup, and preparation of demo environment.

Milestones: System successfully deployed and accessible to users.

Verification: Milestone is reached when the client confirms that the system is operational in the demo environment.

Effort Estimate: 1 week, approximately 40 staff-hours (deployment administrator and database administrator).

6.7 Phase 6: Documentation and Presentation

Tasks: Preparation of technical documentation, user manual, and final presentation.

Deliverables include source code, ERD, UML diagrams, testing reports, and demo slides.

Milestones: Submission of documentation and delivery of final presentation to stakeholders.

Verification: Milestone is reached when the client receives all deliverables and presentation is conducted.

Effort Estimate: 1 week, approximately 40 staff-hours (analyst, project manager, designer).

6.8 Total Effort Estimate

- Requirements Analysis: 80 staff-hours
- System Design: 120 staff-hours
- Implementation: 240 staff-hours
- Testing: 100 staff-hours
- Deployment: 40 staff-hours
- Documentation & Presentation: 40 staff-hours

7. Requirements Analysis and Design

Requirements analysis and system design form the foundation of the University of Eswatini Online Voting System project. These phases ensure that the system is aligned with institutional needs, technically feasible, and secure. By applying structured methods and techniques, the project team captures stakeholder expectations, translates them into technical specifications, and produces design artifacts that guide implementation.

7.1 Methods and Techniques for Requirements Analysis

During requirements analysis, the following methods and techniques will be employed:

- Interviews and Consultations
Direct interviews with administrators, student representatives, and auditors will be conducted to gather functional and non-functional requirements. This ensures that the system reflects actual electoral practices.
- Surveys and Questionnaires
Surveys distributed to students will capture usability expectations, accessibility needs, and preferred features. This technique provides quantitative data to support design decisions.
- Document Review
Existing election policies, student union constitutions, and institutional regulations will be reviewed to ensure compliance with governance structures.
- Use Case Modeling
Use case diagrams will be developed to illustrate interactions between users (voters, administrators, auditors) and the system. This technique clarifies functional requirements and system boundaries.
- Risk Assessment
Early identification of risks such as multiple voting attempts, unauthorized access, and data privacy breaches will be integrated into requirements analysis to shape security specifications.

7.2 Methods and Techniques for System Design

System design translates requirements into technical blueprints using the following techniques:

- Entity-Relationship Diagrams (ERD)
ERDs will be used to design the database schema, defining tables for users, candidates, votes, and results. This ensures data integrity and efficient storage.
- UML Diagrams
Class diagrams, sequence diagrams, and activity diagrams will be created to model system architecture and workflows. These diagrams provide clarity on module interactions.
- Wireframing and Prototyping
Wireframes for login pages, voting interfaces, results dashboards, and administrative panels will be developed to visualize user experience. Prototypes may be tested with sample users to validate usability.
- Three-Tier Architecture Design
The system will adopt a three-tier architecture (presentation, application, database), ensuring scalability, modularity, and maintainability.
- Technology Stack Selection
The design phase will finalize the technology stack, including frontend (HTML, CSS, JavaScript), backend (PHP, Node.js, or Python), and database (MySQL or PostgreSQL).

7.3 Resources and Tools Required

To support these methods and techniques, the following resources and tools will be used:

Software Tools:

- UML modeling tools (Lucidchart, Visual Paradigm) for diagrams.
- Database design tools (MySQL Workbench, pgAdmin) for ERDs.
- Wireframing tools (Figma, Balsamiq) for UI/UX design.
- IDEs (Visual Studio Code, PyCharm) for coding and prototyping.

Hardware Resources:

- University servers for hosting and testing.
- PCs and laptops for development and design activities.

University Resources:

- Access to student records and authentication protocols.
- Policy documents governing elections.
- Sample datasets for testing modules.

8. Implementation

The implementation phase of the University of Eswatini Online Voting System project represents the transition from design to actual development. It is the most resource-intensive stage, spanning four weeks, during which the system's modules are coded, integrated, and tested for functionality. The aim of this phase is to transform design specifications into a working system that meets the requirements identified during analysis.

8.1 Implementation Activities

The implementation phase is divided into several key modules:

- *Authentication Module:* This module manages user registration, login, and password hashing. It enforces secure access and ensures that only authorized voters can participate.
- *Candidate Management Module:* Administrators use this module to add, edit, or delete candidate information. It provides a structured interface for managing electoral data.
- *Voting Module:* The voting module enforces the principle of one-person-one-vote. Once a vote is cast, the system locks the voter's account to prevent multiple submissions.
- *Results Dashboard:* This module provides real-time tallying of votes, displaying percentages and charts to enhance transparency.
- *Administrative Panel:* The admin panel integrates all modules, allowing administrators to manage users, candidates, and results from a centralized interface.

Each module is developed iteratively, tested individually, and then integrated into the overall system architecture.

8.2 Resources and Tools Required

Software Tools

- *Programming Languages:* PHP, Node.js, or Python for backend development; HTML, CSS, and JavaScript for frontend interfaces.
- *Database Management Systems:* MySQL or PostgreSQL for storing and managing voter, candidate, and results data.
- *Integrated Development Environments (IDEs):* Visual Studio Code, PyCharm, or Eclipse to support coding and debugging.
- *Version Control Systems:* GitHub or GitLab for managing source code, tracking changes, and supporting collaboration.
- *Testing Tools:* Selenium, JUnit, or custom scripts for automated testing of modules.

Hardware Resources

- *Servers*: University servers or cloud-based platforms (AWS, Heroku) for hosting the system.
- *Development Machines*: PCs and laptops for programmers, testers, and administrators.
- *Networking Infrastructure*: Secure connections to support deployment and testing environments.

University Resources

- *Authentication Protocols*: Student ID formats and registration data provided by the university.
- *Sample Datasets*: Candidate lists and mock voter records for testing modules.
- *Policy Documents*: Electoral rules and regulations to guide implementation.

8.3 Verification of Implementation

Milestones in the implementation phase are reached when each module is completed and passes initial unit testing. Integration testing ensures that modules interact seamlessly, while client validation confirms that the system reflects actual electoral practices. Progress is tracked using Gantt and PERT charts, ensuring adherence to the critical path.

9. Testing

Testing is a critical phase in the development of the University of Eswatini Online Voting System. It ensures that the system functions as intended, meets security requirements, and provides a user-friendly experience. Testing must be carefully planned, executed in a controlled environment, and documented to validate the reliability and integrity of the system before deployment.

9.1. Testing Environment

Location: Controlled university IT laboratory and simulated online environment.

Network Setup: Secure LAN and WAN configurations to replicate real-world usage.

User Simulation: Test accounts representing students, administrators, and election officers.

Security Controls: Firewalls, intrusion detection systems, and encryption protocols enabled during testing.

9.2. Test Equipment

- *Servers*: Dedicated application and database servers with backup redundancy.
- *Client Devices*: Laptops, desktops, and mobile devices to simulate diverse student access.

9.3. Testing Plan

Testing must be carefully planned to cover functional, performance, and security aspects:

Functional Testing: Verify login, ballot casting, vote tallying, and result display.

Performance Testing: Assess system response under peak loads.

Security Testing: Ensure protection against threats such as SQL injection, phishing, and unauthorized access.

Usability Testing: Confirm that the interface is intuitive and accessible to all students.

9.4. Integration and Testing Order

The order of integration and testing is critical to minimize risks:

- *Database Integration:* Test secure storage and retrieval of voter data.
- *Authentication Module:* Validate student login and identity verification.
- *Voting Interface:* Ensure ballots are displayed correctly and votes are recorded accurately.
- *Vote Tallying Engine:* Test aggregation of votes and generation of preliminary results.
- *Results Publication Module:* Verify secure and transparent display of certified results.
- *Audit and Logging System:* Confirm that all activities are logged for accountability.

9.5. Testing Procedures

Step 1: Define test cases and expected outcomes.

Step 2: Execute unit tests on individual modules.

Step 3: Conduct integration tests to ensure modules work together seamlessly.

Step 4: Perform system-wide tests under simulated election conditions.

Step 5: Document results, identify defects, and apply corrective measures.

Step 6: Conduct final acceptance testing with stakeholders before deployment.

9.6. Testing Strategy

The testing strategy ensures systematic validation of the system's functionality, performance, and security:

Black-Box Testing: Focus on inputs and outputs without examining internal code.

White-Box Testing: Inspect internal logic, algorithms, and code paths for vulnerabilities.

Stress and Load Testing: Push the system beyond expected limits to evaluate stability.

Security Penetration Testing: Simulate attacks to identify weaknesses in authentication, encryption, and data protection.

User Acceptance Testing (UAT): Engage student representatives to validate usability and trustworthiness.

Continuous Testing: Integrate automated testing into the development cycle for ongoing quality assurance.

9.7. Testing schedule

Testing Phase	Activities	Duration	Responsible Team
Unit Testing	Test individual modules (database, authentication, voting interface).	3 days	Developer
Integration Testing	Combine modules and test interactions.	3 days	designer +Developer
System Testing	Full system evaluation under simulated election conditions.	5 days	Whole Team
User Acceptance Testing	Pilot test with student representatives and election officers.	3 days	leader + Student Council

10.Resources

The following table captures the essential hardware, software and human resources needed to execute and sustain the University of Eswatini online voting system

Resources	Type	Quantity	Purpose
Server computer	Hardware	1	Human Resources – Support Staff
Database Management System (MySQL)	Software	1	Stores voter records, ballots, and election results securely.
Web Development Framework (Python Django)	Software	1	Enables development of the front-end and back-end of the system.
SSL Certificates & Encryption Tools	Software	1	SSL Certificates & Encryption Tools
Backup Storage (External Drives/Cloud)	Hardware	2	Maintains copies of data to prevent loss during system failure.
Version Control (GitHub/GitLab)	Software	1	Manages source code and supports collaborative development.
Project Manager	Human Resources	1	Oversees project execution and ensures milestones are met.
Developers	Human Resources	2	Build, test, and maintain the system.
Support Staff	Human Resources	3	Provides training and technical assistance to users.

11. Quality assurance

11.1. **Quality Assurance Organization**

The quality assurance (QA) process will be managed by a dedicated team structured to ensure accountability and transparency.

QA Manager (project manager): Responsible for overseeing compliance with institutional policies and international standards.

Security Analyst (designer): Conduct risk assessments and penetration testing to safeguard sensitive data.

Developer: Implement corrective actions based on QA findings.

11.2. **Procedures to Assure Quality**

The following procedures will be applied to guarantee that the system meets the stated quality requirements:

- *Requirements Verification:* A requirements traceability matrix will be maintained to ensure that all functional and non-functional requirements are tested.
- *Testing Strategy:* User Acceptance Testing (UAT): Controlled testing by students and staff prior to deployment. System Testing: Assessment of the system against overall requirements. Integration Testing: Validation of module interactions. Unit Testing: Verification of individual modules.
- *Security Assurance:* Encryption of votes and student identification data. Role-based access control to prevent unauthorized use. Regular penetration testing to identify vulnerabilities.
- *Performance Assurance:* Load testing to confirm system stability during peak voting periods. Stress testing to evaluate resilience under extreme conditions.
- *Documentation and Reviews:* Detailed test plans, test cases, and results will be maintained. Peer reviews of code and documentation will be conducted. Regular audits will ensure compliance with university IT governance policies.

11.3. **Standards and Tools**

The QA process will adhere to recognized international standards and utilize modern tools:

Standards: ISO/IEC 25010 (software quality model), IEEE standards for software testing.

Tools: Automated testing frameworks like version control systems (Git), and bug tracking platforms (Jira).

11.4. **Configuration Management and Change Control**

Changes during the software development process are inevitable due to evolving requirements, stakeholder feedback, and technological updates. To ensure that these changes are managed systematically, the University of Eswatini Online Voting System will adopt a structured configuration management plan. This plan guarantees that modifications do not compromise quality, security, or system integrity.

12. Change Management

Changes during the software development process are inevitable due to evolving requirements, stakeholder feedback, and technological updates. To ensure that these changes are managed systematically, the University of Eswatini Online Voting System will adopt a structured configuration management plan. This plan guarantees that modifications do not compromise quality, security, or system integrity

12.1. Procedures for Managing Changes

Change Request Submission: All proposed changes must be formally documented in a Change Request Form.

Impact Analysis: Each request is evaluated for its effect on functionality, performance, security, and project timeline.

Implementation: Approved changes are scheduled, coded, and tested

Verification and Validation: Regression testing ensures that the change does not introduce new defects.

Documentation Update: System manuals, user guides, and technical documents are revised to reflect the change

12.2. Separate Configuration Management Plan

Detailed procedures, roles, and responsibilities may be documented in a separate Configuration Management Plan, which will include:

- Change request templates
- Approval workflows
- Versioning policies

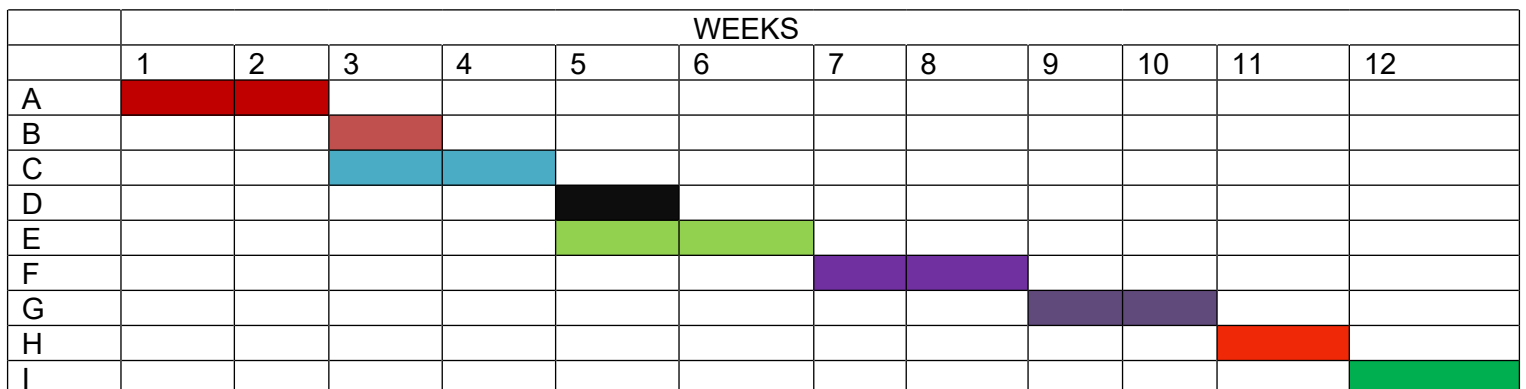
By adopting a clear configuration management framework, the University of Eswatini Online Voting System ensures that changes are handled in a controlled, transparent, and efficient manner. This approach minimizes risks, maintains system stability, and upholds the integrity of the voting process.

Describing the inter-relationships between tasks in a project plan using simple table , GANTT chart and PERT diagrams

1. Using a simple table.

TASK ID	ASSIGNMENT	DURATION	PREDECESSORS	COST PER DAY(E)
A	Gather requirements, define scope, identify stakeholders, draft requirement specification	2 WEEKS	NONE	350
B	Create ERD, design database schema	1 WEEK	A	450
C	Draw UI mockups, define system architecture, select technology stack	2 WEEK	A	400
D	Develop authentication module	1 WEEK	B, C	800
E	Build voting & results system	2 WEEKS	C	950
F	Create admin dashboard & integrate database	2 WEEK	D, E	700
G	Conduct unit tests, integration tests, security checks	2 WEEKS	F	550
H	Host system on server, configure database, ensure secure access	1 WEEK	G	600
I	Write technical documentation, prepare user manual, create demo slides, rehearse presentation	1 WEEK	H, G	300

2. Using a bar chart (Gantt chart)



3. *Using a graph (PERT diagram)*

Critical path: A-C-E-F-G-H-I

Completion Time: 12 weeks

